

# Contents

|          |                                                                                 |          |
|----------|---------------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Log setups</b>                                                               | <b>1</b> |
| 1.1      | Namespace log N1 - Simple namespace creation and deletion . . . . .             | 1        |
| 1.2      | Namespace log N2 - Namespace with resource quotas . . . . .                     | 1        |
| 1.3      | Pod log P1 - Simple Pod creation and inspection . . . . .                       | 1        |
| 1.4      | Pod log P2 - Image download . . . . .                                           | 2        |
| 1.5      | ConfigMap C1 - Create a ConfigMap and use it in a Pod, then delete it . . . . . | 3        |
| 1.6      | Secret S1 - Create a Secret and use it in a Pod, then delete it . . . . .       | 3        |

## 1 Log setups

### 1.1 Namespace log N1 - Simple namespace creation and deletion

- Create a namespace:

```
kubectl create namespace rising
```

- Get the namespace:

```
kubectl get namespaces
```

- Describe the namespace:

```
kubectl describe namespace rising
```

- Delete the namespace:

```
kubectl delete namespace rising
```

### 1.2 Namespace log N2 - Namespace with resource quotas

### 1.3 Pod log P1 - Simple Pod creation and inspection

- Create a pod:

```
cat <<EOF | kubectl apply -f -
  apiVersion: v1
  kind: Pod
  metadata:
    name: rising-pod
    namespace: rising
  spec:
    containers:
    - name: first-container
      image: nginx
    - name: second-container
      image: alpine
      command: ["sh", "-c", "echo Hello, Kubernetes! && sleep 3600"]
EOF
```

- Inspect the Pod:

```
kubectl -n rising describe pod rising-pod
```

- Patch the Pod and try to add a third container, this will fail:

```
kubectl -n rising patch pod rising-pod --type='json' -p='[{"op": "add", "path":
↪  "/spec/containers/1", "value": {"name": "third-container", "image": "nginx"}}]'
```

- Get the pod log:

```
kubectl -n rising logs rising-pod -c second-container
```

- Execute commands in the second containers, the first will complain about the shell, the second will work:

```
kubectl -n rising exec -t rising-pod -c second-container -- /bin/bash -c "echo Hello again,
↳ Kubernetes!"
kubectl -n rising exec -t rising-pod -c second-container -- /bin/sh -c "echo Hello again,
↳ Kubernetes!"
```

- Delete the pod:

```
kubectl -n rising delete pod rising-pod
```

## 1.4 Pod log P2 - Image download

**Prerequisites:** run the following helper script to wipe the image cache on all nodes:

```
for node in $(kubectl get nodes -o jsonpath='{.items[*].metadata.name}'); do
  echo "Pruning images on $node"
  ssh -F ssh/cluster-setup $node "sudo crictl rmi --prune"
done
```

With the image cache cleared, we can now run the following tests:

- Create a Pod with a container that uses a non-existent image:

```
cat <<EOF | kubectl apply -f -
  apiVersion: v1
  kind: Pod
  metadata:
    name: image-pod
    namespace: rising
  spec:
    containers:
      - name: image-container
        image: www.fbk.eu/non-existent-image
        command: ["sh", "-c", "echo Hello, Kubernetes! && sleep 3600"]
EOF
```

- Get the pod log:

```
kubectl -n rising logs image-pod -c image-container
```

- Get the events that led to the failure:

```
kubectl get events -n rising --sort-by='.metadata.creationTimestamp'
```

- Delete the pod:

```
kubectl -n rising delete pod image-pod
```

- Create a pod with an image that will surely not be in cache:

```
cat <<EOF | kubectl apply -f -
  apiVersion: v1
  kind: Pod
  metadata:
    name: image-pod
    namespace: rising
  spec:
    containers:
      - name: image-container
        image: busybox
        command: ["sh", "-c", "echo Hello, Kubernetes! && sleep 3600"]
EOF
```

- Wait for the Pod to be downloaded and monitor the events:

```
kubectl wait --for=condition=Ready pod/image-pod -n rising
kubectl get events -n rising --sort-by='.metadata.creationTimestamp'
```

- Delete the pod:

```
kubectl -n rising delete pod image-pod
```

## 1.5 ConfigMap C1 - Create a ConfigMap and use it in a Pod, then delete it

- Create a ConfigMap:

```
cat <<EOF | kubectl apply -f -
  apiVersion: v1
  kind: ConfigMap
  metadata:
    name: my-config
    namespace: rising
  data:
    status: "decent"
EOF
```

- Get the configmap

```
kubectl -n rising get configmap my-config -o yaml
```

- Deploy a Pod that uses the ConfigMap:

```
cat <<EOF | kubectl apply -f -
  apiVersion: v1
  kind: Pod
  metadata:
    name: config-pod
    namespace: rising
  spec:
    containers:
      - name: config-container
        image: alpine
        command: ["sh", "-c", 'echo Status is ${STATUS} && sleep 3600']
        env:
          - name: STATUS
            valueFrom:
              configMapKeyRef:
                name: my-config
                key: status
EOF
```

- Access the logs of the Pod:

```
kubectl wait --for=condition=Ready pod/config-pod -n rising
kubectl -n rising logs config-pod -c config-container
```

- Clean everything up:

```
kubectl -n rising delete pod config-pod
kubectl -n rising delete configmap my-config
```

## 1.6 Secret S1 - Create a Secret and use it in a Pod, then delete it

- Create a secret

```
cat <<EOF | kubectl apply -f -
  apiVersion: v1
  kind: Secret
  metadata:
    name: my-secret
    namespace: rising
  type: Opaque
  data:
    username: $(echo -n "admin" | base64)
    password: $(echo -n "password" | base64)
EOF
```

- Inspect the secret:

```
kubectl -n rising get secret my-secret -o yaml
```

- Attach the secret to a Pod:

```
cat <<EOF | kubectl apply -f -
  apiVersion: v1
  kind: Pod
  metadata:
    name: secret-pod
    namespace: rising
  spec:
    containers:
      - name: secret-container
        image: alpine
        command: ["sh", "-c", 'echo Username is \${USERNAME} && echo Password is \${PASSWORD} && sleep 3600']
        env:
          - name: USERNAME
            valueFrom:
              secretKeyRef:
                name: my-secret
                key: username
          - name: PASSWORD
            valueFrom:
              secretKeyRef:
                name: my-secret
                key: password
EOF
```

- Inspect the Pod:

```
kubectl -n rising describe pod secret-pod
```

- Access the logs of the Pod:

```
kubectl wait --for=condition=Ready pod/secret-pod -n rising
kubectl -n rising logs secret-pod -c secret-container
```

- Clean everything up:

```
kubectl -n rising delete pod secret-pod
kubectl -n rising delete secret my-secret
```